

PC Build Store Management System

System Analysis

Group Members:

Abdul Hanan (Lead)

Haroon Ikram

Aitzaz

Ahmad

Submission Date: 31st May 2026

1. Feasibility Study

Technical Feasibility

We have all the tools needed. Java 17 is free, XAMPP gives us MySQL for free, NetBeans is free, and Git is free. All team members have computers with these installed.

Operational Feasibility

The system is designed for PC shop owners who may not be very technical. The GUI is simple with clear labels and buttons.

Economic Feasibility

Total cost is zero. Everything uses free and open-source tools.

2. Requirements Analysis

Functional Requirements

- User can browse PC builds filtered by exact budget
- User can add, edit, and delete builds
- User can browse GPU upgrade options
- User can filter GPUs by brand (Nvidia/AMD)
- User can add new GPU options
- User can select a budget and purchase a build
- User can upgrade GPU during purchase
- System generates a bill after purchase
- System tracks total revenue and number of bills
- Dashboard shows real-time statistics
- Purchased builds are removed from available inventory

Non-Functional Requirements

- Application must start within 5 seconds
- Database queries must respond within 1 second
- GUI must be responsive and not freeze
- Data must persist in MySQL database

3. Use Cases

Use Case 1: Browse Builds

Actor: User. The user opens Build Catalog, types a budget in the search box, and the table filters to show only builds at that exact budget.

Use Case 2: Add Build

Actor: User. The user fills in all form fields and clicks "Add Build". The system validates the input and adds the build to the database.

Use Case 3: Purchase Build

Actor: User. The user goes to Billing, selects a budget, clicks "Purchase Build", selects a build, chooses a GPU upgrade (optional), and confirms. The system creates a bill, deletes the build, and updates dashboard stats.

4. Data Flow Analysis

- User enters budget in search field
- BuildDAO.getBuildsByBudget() queries database with exact match
- Results displayed in table
- User selects a row -> form populated with build data
- User selects budget from dropdown in Billing
- User clicks Purchase Build
- BuildDAO.getBuildsByBudget() returns matching builds
- User selects a build from dialog
- GPU options for that budget are shown
- User chooses GPU upgrade or skips
- BillDAO.saveBill() creates bill record
- BuildDAO.deleteBuild() removes build from inventory
- User navigates to Dashboard -> loadStats() updates stats

5. Database Analysis

Table	Rows	Purpose
builds	184	PC build configurations
gpu_options	59	GPU upgrade options
bills	variable	Purchase transactions

6. Risk Analysis

Risk	Impact	Mitigation
Database down	App crashes	Try-catch with error dialogs
Invalid input	Bad data	Input validation in forms
Concurrent access	Data conflict	Single user system

7. Conclusion

The analysis shows the project is feasible, requirements are clear, and the design is sound. All modules integrate properly through the shared database connection.